

OVATIO

Tutoriel d'implementation des nouvelles fonctionnalites

Laravel 12 + React 18

Google OAuth | Apple Sign In | RSS Auto | Traduction des Avis

Mai 2026

Tutoriel ? Implémentation des nouvelles fonctionnalités

Projet OVATIO ? Laravel 12 + React 18

Légende : - [LOCAL] **LOCAL** = à faire sur ta machine de dev - [EXTERNE] **EXTERNE** = sur un site tiers (Google, Apple, DeepL...) - [SERVEUR] **SERVEUR** = sur le serveur de production (VPS, cPanel, etc.) - [!] = étape critique, ne pas sauter

TABLE DES MATIÈRES

- Connexion Google OAuth

- Connexion Apple Sign In
- RSS automatique quotidien/hebdomadaire
- Traduction des avis (Reviews)

1. CONNEXION GOOGLE OAUTH

Prérequis

- Un compte Google (Gmail)
- PHP >= 8.2 (déjà en place)
- Composer installé

ÉTAPE 1 ? Créer le projet Google Cloud

[EXTERNE] Sur <https://console.cloud.google.com>

- Va sur <https://console.cloud.google.com>
- Clique sur **"Sélectionner un projet"** en haut -> **"Nouveau projet"**
- Nom du projet : **ovatio-app** -> **Créer**
- Dans le menu gauche : **API et services** -> **Écran de consentement OAuth**
- Choisis **Externe** -> **Créer**
- Remplis :
 - Nom de l'application : **Ovatio**
 - E-mail d'assistance : ton email
 - E-mail contact développeur : ton email
- Clique **Enregistrer et continuer** (les étapes suivantes peuvent être ignorées pour l'instant)
- Dans le menu gauche : **API et services** -> **Identifiants**
- Clique + **Créer des identifiants** -> **ID client OAuth**
- Type d'application : **Application Web**
- Nom : **Ovatio Web Client**
- **Origines JavaScript autorisées** :

CODE

```
http://localhost:8000
http://localhost:3001
https://TON_DOMAINE.com
```

- **URI de redirection autorisés** :

CODE

```
http://localhost:8000/auth/google/callback
https://TON_DOMAINE.com/auth/google/callback
```

- Clique **Créer**
- [!] **Copie le Client ID et le Client Secret** ? tu en auras besoin à l'étape 3

ÉTAPE 2 ? Installer Laravel Socialite

[LOCAL] **LOCAL ? dans le terminal, à la racine du projet**

BASH

```
composer require laravel/socialite
```

Vérifie que la ligne suivante apparaît dans `composer.json` :

JSON

```
"laravel/socialite": "^5.x"
```

ÉTAPE 3 ? Configurer les variables d'environnement

[LOCAL] LOCAL ? fichier `.env` (à la racine du projet)

Ajoute ces lignes à la fin du fichier `.env` :

ENV

```
GOOGLE_CLIENT_ID=ton_client_id_copié_depuis_google  
GOOGLE_CLIENT_SECRET=ton_client_secret_copié_depuis_google  
GOOGLE_REDIRECT_URI=http://localhost:8000/auth/google/callback
```

Ajoute aussi dans `.env.example` (pour que les autres développeurs sachent que ces variables existent) :

ENV

```
GOOGLE_CLIENT_ID=  
GOOGLE_CLIENT_SECRET=  
GOOGLE_REDIRECT_URI=
```

ÉTAPE 4 ? Configurer le service dans Laravel

[LOCAL] LOCAL ? fichier `config/services.php`

Ajoute ce bloc dans le tableau `return [...]` :

PHP

```
'google' => [  
    'client_id' => env('GOOGLE_CLIENT_ID'),  
    'client_secret' => env('GOOGLE_CLIENT_SECRET'),  
    'redirect' => env('GOOGLE_REDIRECT_URI'),  
],
```

ÉTAPE 5 ? Créer la migration pour les colonnes OAuth

[LOCAL] LOCAL ? dans le terminal

BASH

```
php artisan make:migration add_oauth_columns_to_users_table
```

Ouvre le fichier créé dans `database/migrations/` (le plus récent, nommé `...addoauthcolumnstouserstable.php`) et remplace son contenu par :

PHP

```
<?php  
  
use Illuminate\Database\Migrations\Migration;  
use Illuminate\Database\Schema\Blueprint;  
use Illuminate\Support\Facades\Schema;  
  
return new class extends Migration  
{  
    public function up(): void  
    {  
        Schema::table('users', function (Blueprint $table) {  
            $table->string('provider')->nullable()->after('email');  
            $table->string('provider_id')->nullable()->after('provider');  
            $table->string('password')->nullable()->change(); // nullable car OAuth n'a pas de mot d...  
        });  
    }  
};
```

```
}

public function down(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn(['provider', 'provider_id']);
        $table->string('password')->nullable(false)->change();
    });
}
};
```

Lance la migration :

BASH

```
php artisan migrate
```

ÉTAPE 6 ? Mettre à jour le modèle User

[LOCAL] LOCAL ? fichier `app/Models/User.php`

Dans le tableau `$fillable` , ajoute `'provider'` et `'provider_id'` :

PHP

```
protected $fillable = [
    'login',
    'firstname',
    'lastname',
    'name',
    'email',
    'password',
    'langue',
    'photo',
    'is_disabled',
    'provider', // ← ajouter
    'provider_id', // ← ajouter
];
```

ÉTAPE 7 ? Créer le contrôleur Socialite

[LOCAL] LOCAL ? dans le terminal

BASH

```
php artisan make:controller Auth/SocialiteController
```

Ouvre `app/Http/Controllers/Auth/SocialiteController.php` et remplace tout le contenu par :

PHP

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\Role;
use App\Models\User;
use Illuminate\Support\Facades\Auth;
use Laravel\Socialite\Facades\Socialite;

class SocialiteController extends Controller
{
    // Redirige vers Google
    public function redirectGoogle()
    {
        return Socialite::driver('google')->stateless()->redirect();
    }
}
```

```
}

// Google revient ici après connexion
public function callbackGoogle()
{
    try {
        $googleUser = Socialite::driver('google')->stateless()->user();
    } catch (\Exception $e) {
        return response()->json(['message' => 'Échec de la connexion Google'], 401);
    }

    // Cherche un user existant avec ce Google ID
    $user = User::where('provider', 'google')
        ->where('provider_id', $googleUser->getId())
        ->first();

    // Sinon, cherche par email
    if (!$user) {
        $user = User::where('email', $googleUser->getEmail())->first();
    }

    // Sinon, crée un nouveau compte
    if (!$user) {
        $user = User::create([
            'name' => $googleUser->getName(),
            'firstname' => $googleUser->user['given_name'] ?? $googleUser->getName(),
            'lastname' => $googleUser->user['family_name'] ?? '',
            'login' => $googleUser->getEmail(),
            'email' => $googleUser->getEmail(),
            'photo' => $googleUser->getAvatar(),
            'provider' => 'google',
            'provider_id' => $googleUser->getId(),
            'password' => null,
        ]);

        // Assigne le rôle membre par défaut
        $memberRole = Role::where('role', 'member')->first();
        if ($memberRole) {
            $user->roles()->attach($memberRole->id);
        }
    } else {
        // Met à jour le provider si trouvé par email
        $user->update([
            'provider' => 'google',
            'provider_id' => $googleUser->getId(),
        ]);
    }

    // Crée un token Sanctum et le retourne au frontend
    $token = $user->createToken('google-token')->plainTextToken;

    // Redirige vers le frontend avec le token
    $frontendUrl = config('app.frontend_url', env('FRONTEND_URL', 'http://localhost:3001'));

    return redirect()->away("{$frontendUrl}/auth/callback?token={$token}&user=" . urlencode(json...
        'id' => $user->id,
        'name' => $user->name,
        'email' => $user->email,
        'photo' => $user->photo,
    ]));
}
}
```

ÉTAPE 8 ? Ajouter les routes

[LOCAL] **LOCAL ?** fichier `routes/api.php`

Ajoute ces lignes dans la section des routes publiques (avant le middleware `auth:sanctum`) :

PHP

```
// OAuth Google
Route::get('/auth/google', [App\Http\Controllers\Auth\SocialiteController::class, 'redirect...']);
Route::get('/auth/google/callback', [App\Http\Controllers\Auth\SocialiteController::class, 'callback...']);
```

ÉTAPE 9 ? Côté frontend React

[LOCAL] **LOCAL ?** dans `ovatio-frontend/src/`

Crée un bouton de connexion dans ta page de login. Par exemple dans `ovatio-frontend/src/pages/Login.jsx` (ou l'équivalent) :

JSX

```
const handleGoogleLogin = () => {
  // Redirige vers le backend qui redirige vers Google
  window.location.href = 'http://localhost:8000/api/auth/google';
};

// Dans le JSX :
<button onClick={handleGoogleLogin} className="btn-google">
  Se connecter avec Google
</button>
```

Crée une page `ovatio-frontend/src/pages/AuthCallback.jsx` pour récupérer le token au retour :

JSX

```
import { useEffect } from 'react';
import { useNavigate, useSearchParams } from 'react-router-dom';

export default function AuthCallback() {
  const [params] = useSearchParams();
  const navigate = useNavigate();

  useEffect(() => {
    const token = params.get('token');
    const user = params.get('user');

    if (token) {
      localStorage.setItem('token', token);
      localStorage.setItem('user', user);
      navigate('/dashboard'); // ou ta page d'accueil
    } else {
      navigate('/login');
    }
  }, []);

  return <p>Connexion en cours...</p>;
}
```

Ajoute cette route dans ton `App.jsx` ou router :

JSX

```
<Route path="/auth/callback" element={<AuthCallback />} />
```

ÉTAPE 10 ? Tester en local

[LOCAL] **LOCAL**

BASH

```
# Terminal 1 – lancer le backend
php artisan serve

# Terminal 2 – lancer le frontend
cd ovatio-frontend
npm run dev
```

- Va sur <http://localhost:3001/login>
- Clique "Se connecter avec Google"
- Connecte-toi avec un compte Google
- Tu dois être redirigé sur <http://localhost:3001/auth/callback?token=xxx>
- Vérifie dans la base de données : un user doit être créé avec `provider=google`

2. CONNEXION APPLE SIGN IN

[!] *Apple Sign In est plus complexe que Google. Il faut obligatoirement un **compte Apple Developer payant (\$99/an)**. Sans ça, impossible de continuer.*

Prérequis

- Compte Apple Developer actif sur <https://developer.apple.com>
- Un domaine HTTPS (Apple refuse localhost en production)

ÉTAPE 1 ? Configurer sur Apple Developer

[EXTERNE] Sur <https://developer.apple.com>

A) Créer un App ID :

- Connecte-toi sur <https://developer.apple.com/account>
- Menu : **Certificates, Identifiers & Profiles** -> **Identifiers**
- Clique + -> sélectionne **App IDs** -> **App**
- Description : [Ovatio App](#)
- Bundle ID : [com.ovatio.app](#) (format reverse-domain)
- Dans **Capabilities**, coche **Sign In with Apple** -> **Enable as a primary App ID**
- Clique **Continue** -> **Register**

B) Créer un Service ID :

- **Identifiers** -> + -> **Services IDs**
- Description : [Ovatio Web](#)
- Identifier : [com.ovatio.web](#) ? c'est ton `APPLECLIENTID`
- Clique **Continue** -> **Register**
- Clique sur le Service ID que tu viens de créer
- Coche **Sign In with Apple** -> **Configure**
- Primary App ID : sélectionne [Ovatio App](#)
- **Domains and Subdomains** : [ton-domaine.com](#)
- **Return URLs** : <https://ton-domaine.com/auth/apple/callback>
- **Save** -> **Continue** -> **Register**

C) Créer une clé privée :

- Menu : **Keys** -> +
- Nom : [Ovatio Sign In Key](#)

- Coche **Sign in with Apple** -> **Configure** -> sélectionne ton App ID
- **Register** -> **Download** ? [!] télécharge le fichier **.p8** UNE SEULE FOIS
- Note le **Key ID** affiché (ex: **ABC123DEF4**)
- Note ton **Team ID** visible en haut à droite de ton compte

ÉTAPE 2 ? Installer le provider Apple pour Socialite

[LOCAL] LOCAL ? terminal

BASH

```
composer require socialiteproviders/apple
```

ÉTAPE 3 ? Placer la clé privée Apple

[LOCAL] LOCAL

- Place le fichier **.p8** téléchargé dans **storage/app/** :

CODE

```
storage/app/apple_sign_in.p8
```

- [!] Ajoute ce fichier dans **.gitignore** (ne jamais committer une clé privée) :

CODE

```
/storage/app/apple_sign_in.p8
```

ÉTAPE 4 ? Variables d'environnement Apple

[LOCAL] LOCAL ? fichier **.env**

ENV

```
APPLE_CLIENT_ID=com.ovatio.web
APPLE_CLIENT_SECRET= # laisse vide, sera généré dynamiquement
APPLE_REDIRECT_URI=https://ton-domaine.com/auth/apple/callback
APPLE_TEAM_ID=TONTTEAMID # visible sur developer.apple.com en haut à droite
APPLE_KEY_ID=ABC123DEF4 # le Key ID noté à l'étape 1C
APPLE_PRIVATE_KEY_PATH=storage/app/apple_sign_in.p8
```

ÉTAPE 5 ? Générer le Client Secret Apple (JWT)

[LOCAL] LOCAL ? fichier **app/Services/AppleClientSecretService.php** (créer ce fichier)

PHP

```
<?php

namespace App\Services;

use Lcobucci\JWT\Configuration;
use Lcobucci\JWT\Signer\Ecdsa\Sha256;
use Lcobucci\JWT\Signer\Key\InMemory;
use DateTimeImmutable;

class AppleClientSecretService
{
    public static function generate(): string
    {
        $privateKey = InMemory::file(storage_path('app/' . basename(env('APPLE_PRIVATE_KEY_PATH')))...

        $config = Configuration::forAsymmetricSigner(
```



```

        new Sha256(),
        $privateKey,
        InMemory::plainText('')
    );

    $now = new DateTimeImmutable();

    $token = $config->builder()
        ->issuedBy(env('APPLE_TEAM_ID'))
        ->issuedAt($now)
        ->expiresAt($now->modify('+6 months'))
        ->permittedFor('https://appleid.apple.com')
        ->withHeader('kid', env('APPLE_KEY_ID'))
        ->relatedTo(env('APPLE_CLIENT_ID'))
        ->getToken($config->signer(), $config->signingKey());

    return $token->toString();
}
}

```

Pour que ça fonctionne, installe la librairie JWT :

BASH

```
composer require lcobucci/jwt
```

ÉTAPE 6 ? Configurer le service Apple

[LOCAL] **LOCAL ?** [config/services.php](#)

PHP

```

'apple' => [
    'client_id' => env('APPLE_CLIENT_ID'),
    'client_secret' => \App\Services\AppleClientSecretService::generate(),
    'redirect' => env('APPLE_REDIRECT_URI'),
],

```

[!] Si cette ligne cause des erreurs lors du `php artisan config:cache`, génère le secret dans le contrôleur à la place (voir étape suivante).

ÉTAPE 7 ? Ajouter Apple au contrôleur Socialite

[LOCAL] **LOCAL ?** [app/Http/Controllers/Auth/SocialiteController.php](#)

Ajoute ces deux méthodes à la suite de celles de Google :

PHP

```

// Redirige vers Apple
public function redirectApple()
{
    // Génère le client secret dynamiquement
    config(['services.apple.client_secret' => \App\Services\AppleClientSecretService::generate()]);

    return Socialite::driver('apple')->stateless()->redirect();
}

// Apple revient ici après connexion
public function callbackApple()
{
    config(['services.apple.client_secret' => \App\Services\AppleClientSecretService::generate()]);

    try {
        $appleUser = Socialite::driver('apple')->stateless()->user();
    } catch (\Exception $e) {

```


Ou dans `config/app.php` , dans `providers` :

PHP

```
\SocialiteProviders\Manager\ServiceProvider::class,
```

ÉTAPE 10 ? Bouton "Sign in with Apple" dans React

[LOCAL] LOCAL ? `ovatio-frontend/src/`

Apple impose un style précis pour son bouton. Utilise la librairie officielle :

BASH

```
cd ovatio-frontend
npm install react-apple-signin-auth
```

Dans ta page de login :

JSX

```
import AppleSignin from 'react-apple-signin-auth';

<AppleSignin
  authOptions={{
    clientId: 'com.ovatio.web',
    scope: 'email name',
    redirectURI: 'https://ton-domaine.com/auth/apple/callback',
    usePopup: false,
  }}
  onSuccess={(response) => {
    // En mode non-popup, Apple redirige directement
    console.log(response);
  }}
  render={(props) => (
    <button {...props} className="btn-apple">
      Se connecter avec Apple
    </button>
  )}
/>
```

3. RSS AUTOMATIQUE

L'endpoint `/api/rss` **existe déjà** dans le projet. Cette section ajoute l'envoi automatique par email.

ÉTAPE 1 ? Créer la commande Artisan

[LOCAL] LOCAL ? `terminal`

BASH

```
php artisan make:command SendRssFeed
```

Ouvre `app/Console/Commands/SendRssFeed.php` et remplace le contenu par :

PHP

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Mail;
```

```

use App\Models\User;

class SendRssFeed extends Command
{
    protected $signature = 'rss:send [--mode=latest : latest ou upcoming] [--freq=daily : daily ou w...';
    protected $description = 'Envoie le flux RSS par email aux abonnés';

    public function handle(): void
    {
        $mode = $this->option('mode');
        $freq = $this->option('freq');

        // Récupère le flux RSS depuis l'endpoint existant
        $baseUrl = config('app.url');
        $rssUrl = "{$baseUrl}/api/rss?mode={$mode}";
        $response = Http::get($rssUrl);

        if (!$response->successful()) {
            $this->error("Impossible de récupérer le flux RSS : {$rssUrl}");
            return;
        }

        $rssContent = $response->body();

        // Récupère tous les utilisateurs qui veulent recevoir le RSS
        // (Pour l'instant tous les users – tu pourras ajouter une colonne `rss_subscribed` plus tar...
        $users = User::whereNotNull('email')->get();

        foreach ($users as $user) {
            Mail::send('emails.rss_digest', [
                'user' => $user,
                'rssContent' => $rssContent,
                'mode' => $mode,
                'freq' => $freq,
            ], function ($message) use ($user, $freq) {
                $label = $freq === 'weekly' ? 'hebdomadaire' : 'quotidien';
                $message->to($user->email)
                    ->subject("Ovatio – Résumé {$label} des spectacles");
            });
        }

        $this->info("RSS envoyé à {$users->count()} utilisateurs.");
    }
}

```

ÉTAPE 2 ? Créer le template email RSS

[LOCAL] LOCAL ? créer le fichier `resources/views/emails/rss_digest.blade.php`

HTML

```

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <style>
        body { font-family: Arial, sans-serif; background: #f4f4f4; }
        .container { max-width: 600px; margin: 0 auto; background: white; padding: 30px; }
        h1 { color: #333; }
        .label { background: #6c3cde; color: white; padding: 4px 10px; border-radius: 4px; font-size...
    </style>
</head>
<body>
    <div class="container">
        <h1>Bonjour {{ $user->firstname ?? $user->name }},</h1>

```

```

@if($freq === 'weekly')
    <p>Voici votre résumé <strong>hebdomadaire</strong> des spectacles Ovation.</p>
@else
    <p>Voici votre résumé <strong>quotidien</strong> des spectacles Ovation.</p>
@endif

<p>
    <span class="label">{{ $mode === 'upcoming' ? 'À venir' : 'Dernières nouveautés' }}</span>
</p>

<p>Retrouvez le flux RSS complet ici :</p>
<p><a href="{{ config('app.url') }}/api/rss?mode={{ $mode }}">
    {{ config('app.url') }}/api/rss?mode={{ $mode }}
</a></p>

<hr>
<p style="font-size:12px; color:#999;">
    Vous recevez cet email car vous êtes inscrit sur Ovation.
    Pour vous désinscrire, modifiez vos préférences dans votre profil.
</p>
</div>
</body>
</html>

```

ÉTAPE 3 ? Planifier l'envoi automatique

[LOCAL] LOCAL ? fichier `routes/console.php`

Ajoute ces lignes :

PHP

```

use Illuminate\Support\Facades\Schedule;

// Envoi quotidien à 8h00
Schedule::command('rss:send --mode=latest --freq=daily')->dailyAt('08:00');

// Envoi hebdomadaire le lundi à 9h00
Schedule::command('rss:send --mode=upcoming --freq=weekly')->weeklyOn(1, '09:00');

```

ÉTAPE 4 ? Configurer le CRON sur le serveur

[SERVEUR] SERVEUR ? accès SSH ou cPanel

Sur le serveur de production, le scheduler Laravel doit tourner toutes les minutes.

Via SSH :

BASH

```
crontab -e
```

Ajoute cette ligne (remplace le chemin par le vrai chemin de ton projet) :

CODE

```
* * * * * cd /var/www/ton-projet && php artisan schedule:run >> /dev/null 2>&1
```

Via cPanel :

- Connecte-toi à cPanel
- Section **Cron Jobs**
- Fréquence : **Every Minute** (*)
- Commande : `php /home/toncompte/public_html/artisan schedule:run`

ÉTAPE 5 ? Tester en local

[LOCAL] LOCAL

BASH

```
# Test manuel de la commande
php artisan rss:send --mode=latest --freq=daily

# Test du scheduler (simule une minute d'horloge)
php artisan schedule:run

# Voir les emails dans les logs (si MAIL_MAILER=log dans .env)
tail -f storage/logs/laravel.log
```

CONFIGURATION MAIL (pour que les emails partent vraiment)

[LOCAL] LOCAL / [SERVEUR] SERVEUR ? fichier `.env`

Pour les tests locaux, utilise **Mailtrap** (gratuit) :

- Va sur <https://mailtrap.io> -> crée un compte gratuit
- Inbox -> SMTP Settings -> Laravel -> copie les credentials
- Mets à jour `.env` :

ENV

```
MAIL_MAILER=smtp
MAIL_HOST=sandbox.smtp.mailtrap.io
MAIL_PORT=2525
MAIL_USERNAME=ton_username_mailtrap
MAIL_PASSWORD=ton_password_mailtrap
MAIL_FROM_ADDRESS="noreply@ovatio.be"
MAIL_FROM_NAME="Ovatio"
```

En production : remplace par les credentials de ton vrai serveur SMTP (Gmail, Mailgun, Resend...).

4. TRADUCTION DES AVIS (REVIEWS)

Cette section sera complétée une fois l'API de traduction choisie (DeepL recommandé). Structure prévue : [GET /api/reviews/{id}/translate?lang=fr](#)

Choix recommandé : DeepL

- Gratuit jusqu'à 500 000 caractères/mois
- Inscription : <https://www.deepl.com/fr/pro-api> (plan Free)
- Récupère ta **clé API DeepL** depuis le tableau de bord

Structure prévue (à implémenter quand l'API est choisie)

Fichiers à créer :

CODE

```
app/Services/TranslationService.php
app/Http/Controllers/ReviewTranslationController.php
```

Route à ajouter dans `routes/api.php` :

PHP

```
Route::get('/reviews/{id}/translate', [ReviewTranslationController::class, 'translate']);
```

Variable d'environnement à ajouter dans `.env` :

```
ENV
DEEPL_API_KEY=ta_cle_deepl
# ou
GOOGLE_TRANSLATE_KEY=ta_cle_google
```

Reviens ici une fois l'API décidée pour que le code soit généré.

RÉSUMÉ ? CE QUI SE FAIT OÙ

Fonctionnalité	Local [LOCAL]	Externe [EXTERNE]	Serveur [SERVEUR]
Google OAuth	composer, code, migration, test	Google Cloud Console	.env prod, domaine HTTPS
Apple Sign In	composer, code, clé .p8	Apple Developer (\$99/an)	.env prod, domaine HTTPS obligatoire
RSS automatique	commande, template email, schedule	Mailtrap (test)	crontab *
Traduction avis	service, controller, route	DeepL/Google/Libre	.env prod avec la clé API

ORDRE DE TRAVAIL RECOMMANDÉ

```
CODE
Semaine 1 → Branch feature/google-oauth (le plus simple, bon départ)
Semaine 1 → Branch feature/rss-scheduler (peut tourner en parallèle)
Semaine 2 → Branch feature/apple-signin (après Google, base identique)
Semaine 2 → Branch feature/review-translation (après décision sur l'API)
```

Tutoriel généré pour le projet OVATIO ? Laravel 12 + React 18 ? Mai 2026